

LABORATORIO 10

¿Y a las tres de la mañana?

*Kafbat UI y REST Proxy, y por qué el tablero no es el clúster.
Guía para hacerlo por tu cuenta, paso a paso*

Confluent Platform 8.2.0 · 3 brokers KRaft · REST Proxy · Kafbat UI

Duración estimada · 40 minutos

NovaTech Logistics · caso de estudio

Antes de empezar

Qué vas a lograr

Éste es el único laboratorio del curso que se hace mirando, no tecleando. Vas a poner lado a lado la salida de un comando y la pantalla que dibuja ese mismo dato, y vas a comprobar que son los mismos números.

Y termina con una advertencia, porque **una interfaz bonita es la forma más rápida de perder la costumbre de la línea de comandos**, que es lo único que siempre va a estar.

LO QUE ESTE LABORATORIO DEMUESTRA

Todo lo que viste por consola está aquí dibujado, y aun así la consola es lo que vas a tener a las tres de la mañana.

Tres partes, y las tres se ven en pantalla.

La parte	Dónde se comprueba
Está aquí dibujado	Paso 2. Cada campo del tablero tiene su comando al lado
Dibujado mejor	Paso 3. Hay una cosa que el tablero hace de verdad mejor, y es honesto reconocerla
Y aun así	Paso 4. Apagas el tablero y el clúster no se entera

Qué necesitas tener listo

Requisito	Cómo lo compruebas
Docker Desktop corriendo, con 6 GB de RAM	Ajustes de Docker Desktop, pestaña <i>Resources</i>
Git Bash abierto en la carpeta del lab	<code>cd Capitulo_4/lab-10-rest-proxy-kafbat-ui</code>
Un navegador	Cualquiera. Hoy se usa de verdad, no de adorno
Los puertos 9092, 9093, 9094, 8082 y 8090 libres	El paso 1 te avisa si no lo están. Este lab levanta cinco contenedores , dos más que los anteriores
Haber hecho los laboratorios 05 y 06	Hoy se comparan sus salidas contra las pantallas

UN AVISO SOBRE LOS NÚMEROS DE PARTICIÓN DE ESTA GUÍA

Los tres mensajes que siembra el arranque van **sin clave**, así que el productor elige una partición y mete los tres ahí. **Cuál elige cambia en cada arranque.**

Las salidas de esta guía son de una corrida real en la que tocó la **partición 0**. En otras corridas medidas tocó la 2. **Si a ti te toca otra, no está roto**. Lo único que importa es que dos particiones queden en cero y una con todo. De ahí en adelante, todos los números de partición de esta guía se te corren a la tuya.

El caso

Llevas cinco laboratorios leyendo tablas de texto separadas por tabuladores. Es lento, es incómodo, y hay algo que no has podido hacer ni una vez. **Mirar el clúster entero a la vez.**

Cuando alguien en **NovaTech Logistics** pregunta cómo va todo, no hay un comando que conteste eso. Hay que preguntar tópico por tópico, grupo por grupo, y armar el cuadro en la cabeza. Para un clúster con cuatro tópicos se puede. Para uno con doscientos, no.

Y ahí aparece la solución obvia con la trampa que trae puesta.

La solución obvia. Una interfaz web que lo dibuje. Existe, está en este laboratorio, y es buenísima.

La trampa. Que el equipo se acostumbre a que el clúster es lo que se ve en esa pantalla. Y entonces llega el día en que hay un incidente a las tres de la mañana, y alguien entra por SSH a un servidor donde no hay navegador, ni túnel, ni la interfaz levantada. Y no sabe qué escribir.

Hay una segunda mitad, más incómoda. **La interfaz es una cosa más que se puede caer.** Es un contenedor con su propia memoria, su propia versión y su propia forma de romperse. Un tablero caído se parece muchísimo a un clúster caído, si no sabes distinguirlos. **El paso 4 los distingue en vivo.**

LA METÁFORA · HOY ENTRAN EL PIZARRÓN Y LA VENTANILLA

Kafka sigue siendo **el restaurante**. El **mozo** es el broker, el **tipo de comanda** es el tópico, los **sectores del salón** son las particiones, las **libretas de respaldo** son las réplicas y la **brigada** es el grupo de consumidores.

Hoy entran dos piezas nuevas, y hacen cosas opuestas.

El pizarrón del salón. El tablero donde el jefe de turno ve de un vistazo qué sectores están cubiertos y cuál se está atrasando. Es **Kafbat UI**.

La ventanilla de la calle. Por donde un repartidor de afuera deja un pedido sin entrar al local ni conocer a nadie. Es el **REST Proxy**.

Lo que importa del pizarrón, y es el corazón de este laboratorio. **El pizarrón no atiende mesas.** No cocina, no lleva platos, no cobra. Lo único que hace es decirte a qué mesa correr.

Un jefe de turno que mira el pizarrón y entiende que la mesa 12 lleva veinte minutos esperando es un buen jefe de turno. Uno que se queda mirando el pizarrón, no.

Y si se borra el pizarrón, el restaurante sigue funcionando exactamente igual. Lo único que se perdió es tu forma cómoda de mirarlo.

LA REGLA QUE SALE DE AHÍ

El tablero no es el clúster. **Es una vista del clúster.** Todo lo que muestra se lo preguntó a los brokers en el momento de dibujarlo.

Qué hay que mirar, antes de mirarlo

Qué vamos a hacer

Levantar el laboratorio y saber exactamente qué hay en el tópico **antes** de abrir el navegador. Si no, el tablero solo va a ser bonito.

Levantar el laboratorio

```
cd Capitulo_4/lab-10-rest-proxy-kafbat-ui
chmod +x bin/*.sh rest-cli/*.sh
bin/start-lab.sh
```

`chmod +x bin/*.sh rest-cli/*.sh` — permiso de ejecución para los dos grupos de scripts. **Este lab tiene una carpeta más que los anteriores**, que es `rest-cli/`

`bin/start-lab.sh` — levanta **cinco contenedores**, o sea tres brokers, el REST Proxy y Kafbat UI. Crea el tópico y le mete tres pedidos de muestra **por HTTP**

EL FINAL DE LO QUE VAS A VER · RECORTADO

```
[5/5] Inicializando tópico y produciendo muestra vía HTTP...
[OK] Clúster disponible
Created topic novatech.lab10.pedidos.
✓ Tópico 'novatech.lab10.pedidos' creado (3 particiones, RF 3)
  ✓ 3 mensajes de muestra enviados por el REST Proxy
```

CLÚSTER NOVATECH LAB 10 OPERATIVO

Componentes:

Broker 1:	localhost:9092
Broker 2:	localhost:9093
Broker 3:	localhost:9094
REST Proxy:	http://localhost:8082
Kafbat UI:	http://localhost:8090

CONCEPTO · PARTICIÓN

Cada uno de los pedazos en que se corta un tópico. Éste tiene **tres**. Son los sectores del salón, y por eso el trabajo se puede repartir entre varios cocineros.

El número de orden de un mensaje dentro de su partición. Empieza en 0 y solo sube. **El offset más nuevo es, en la práctica, la cuenta de mensajes de esa partición.**

Qué forma tiene el tópico

```
docker exec kafka-broker-1 kafka-topics \
  --bootstrap-server kafka-broker-1:29092 \
  --describe --topic novatech.lab10.pedidos
```

docker exec — sin `-i` ni `-t`, porque no le vas a escribir nada al comando. **Solo pregunta y escupe**

--describe — la ficha del tópico. Particiones, factor de réplica, quién es líder de cada una y qué réplicas están al día

LO QUE VAS A VER

```
Topic: novatech.lab10.pedidos   TopicId: yzdXUP9zTqeO7Z-sVv3NYA PartitionCount: 3
ReplicationFactor: 3           Configs: min.insync.replicas=2
    Topic: novatech.lab10.pedidos Partition: 0   Leader: 2       Replicas: 2,3,1 Isr: 2,3,1
Elr:   LastKnownElr:
    Topic: novatech.lab10.pedidos Partition: 1   Leader: 3       Replicas: 3,1,2 Isr: 3,1,2
Elr:   LastKnownElr:
    Topic: novatech.lab10.pedidos Partition: 2   Leader: 1       Replicas: 1,2,3 Isr: 1,2,3
Elr:   LastKnownElr:
```

Es la misma salida del laboratorio 05, y hoy se usa de otra manera. **Como la respuesta correcta contra la que vamos a comparar el tablero.** Quédate con tres cosas. Tres particiones, factor de réplica 3, y las tres con sus tres réplicas en ISR.

Y dónde están los mensajes

```
docker exec kafka-broker-1 kafka-get-offsets \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab10.pedidos --time -1
```

kafka-get-offsets — pregunta por los offsets de cada partición. **Es el comando más barato que hay para saber si un clúster está vivo**, y lo vas a volver a usar en el paso 4

--time -1 — el offset **más nuevo**, o sea el final del tópico. Con `-2` pedirías el más viejo

LO QUE VAS A VER

```
novatech.lab10.pedidos:0:3
novatech.lab10.pedidos:1:0
novatech.lab10.pedidos:2:0
```

DETENTE AQUÍ, PORQUE ESTO ES MÁS INTERESANTE DE LO QUE PARECE

Hay tres mensajes, y **los tres están en la misma partición**. Las otras dos están vacías.

No es un error. Los tres se escribieron **sin clave**, y un productor sin clave se pega a una partición durante su tanda. **En tu corrida la partición llena puede ser otra.**

La pregunta que vale, y que el tablero va a contestar de un vistazo. En un tópico de doscientas particiones, ¿cómo te enterarías tú de que ciento noventa están vacías?

VAS BIEN SI...

Hay **cinco** contenedores arriba, no cuatro. Compruébalo:

```
docker ps --filter "label=com.docker.compose.project=novatech-lab10" \
  --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
```

NAMES	STATUS	PORTS
kafka-rest	Up 17 seconds (healthy)	0.0.0.0:8082->8082/tcp, [::]:8082->8082/tcp
kafbat-ui	Up 17 seconds	0.0.0.0:8090->8080/tcp, [::]:8090->8080/tcp
kafka-broker-1	Up 24 seconds (healthy)	0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
kafka-broker-2	Up 24 seconds (healthy)	0.0.0.0:9093->9093/tcp, [::]:9093->9093/tcp
kafka-broker-3	Up 24 seconds (healthy)	0.0.0.0:9094->9094/tcp, [::]:9094->9094/tcp

Y en el `kafka-get-offsets` **dos particiones están en 0 y una tiene 3**. Anota cuál es la tuya, porque la vas a usar en los tres pasos que siguen.

SI DICE QUE EL PUERTO 8082 O EL 8090 YA ESTÁ EN USO

El síntoma nombra `port is already allocated`. Casi siempre es otro laboratorio del curso que quedó arriba.

```
docker ps --filter "label=com.docker.compose.project" \
  --format '{{.Names}}\t{{.Label "com.docker.compose.project"}}'
```

SI DICE REST PROXY AÚN NO RESPONDE

El REST Proxy arranca después de los brokers y a veces tarda más de los 60 segundos que espera el script. **No es fatal**. Espera medio minuto y comprueba a mano con `curl -s http://localhost:8082/topics`. Si contesta una lista, está bien y puedes seguir.

SI EL `KAFKA-GET-OFFSETS` TE DA LAS TRES PARTICIONES EN 0

Los tres mensajes de muestra no se sembraron. Mándalos tú:

```
rest-cli/rest-produce.sh novatech.lab10.pedidos
'{"pedido":1,"cliente":"ACME","monto":1500}'
```

EL  VERDE DE `REST-PRODUCE.SH` NO PRUEBA QUE EL MENSAJE LLEGÓ

Este envoltorio siempre dice que sí. Imprime  Mensaje enviado vía HTTP y termina en éxito aunque el REST Proxy haya rechazado el envío. **Está medido.** Si te equivocas al escribir el nombre del tópico, el comando tarda un minuto y luego contesta esto:

```
{"offsets":[{"partition":null,"offset":null,"error_code":50003,"error":"Topic
novatech.topico.que.no.existe not present in metadata after 60000
ms."}], "key_schema_id":null, "value_schema_id":null}
✓ Mensaje enviado vía HTTP
```

Y si el JSON del segundo argumento está mal escrito, contesta esto otro, también con su  debajo:

```
{"error_code":400,"message":"Unrecognized token 'ESTO_NO_ES_JSON': was expecting (JSON String,
Number, Array, Object or token 'null', 'true' or 'false')"}
✓ Mensaje enviado vía HTTP
```

En los dos casos el mensaje se perdió y el envoltorio informó éxito. No es un defecto del REST Proxy, que contestó la verdad: es que el envoltorio no mira la respuesta antes de felicitarte.

Así que no leas el  . Lee la línea de arriba. Cuando el envío es bueno trae un `offset` con un número y `error_code: null` , así:

```
{"offsets":[{"partition":0,"offset":3,"error_code":null,"error":null}]...
```

Si ves un `error_code` con número, no llegó nada, por muy verde que esté el  . Es la misma regla del laboratorio 09: cuando automatices envíos, comprueba el offset, no el código de salida.

PASO 2 DE 5

La misma verdad, dos veces

Qué vamos a hacer

Ahora se abre el navegador. **No es un paso decorativo.** Vas a comparar campo por campo con lo que acabas de leer en la consola.

Abre el tablero

```
http://localhost:8090
```

EL PUERTO ES EL 8090, NO EL 8080

Por dentro el contenedor escucha en el 8080, y el `docker-compose.yml` lo publica en el 8090 para no chocar con cualquier otra cosa que uses. **Si escribes 8080 no hay nadie.** Lo ves en la columna PORTS del paso 1, donde dice `0.0.0.0:8090->8080/tcp`. El número de la izquierda es el de tu máquina.

Qué se ve, y con qué comando lo comprobaste

Dónde hacer clic	Qué buscar	Con qué lo comprobaste
Dashboard , la pantalla de entrada	Arriba, Online · 1 clusters . Y una fila <code>novatech-cluster</code> con <i>Version</i> <code>4.2-IV1</code> y <i>Brokers count</i> 3	<code>bin/90-test-lab.sh</code>
Brokers , en el menú de la izquierda	Tres filas, con <i>Broker ID</i> 1, 2 y 3, y sus puertos internos 29092, 29093 y 29094	Los tres <code>healthy</code> del <code>docker ps</code>
Arriba de esa misma pantalla	URP 0 y <i>Controller Type</i> KRaft	Los <code>Isr</code> completos del <code>--describe</code>
Topics y luego <code>novatech.lab10.pedidos</code>	En la pestaña Overview , arriba, Partitions 3 y Replication Factor 3	La primera línea del <code>--describe</code>
Más abajo, en esa misma pestaña Overview	Tres filas, con <i>Partition ID</i> , <i>Replicas</i> y <i>Message Count</i>	Las tres líneas <code>Partition:</code>
La columna Message Count	Dos particiones en 0 y una en 3	El <code>kafka-get-offsets</code>
Pestaña Messages	Los tres pedidos, como JSON legible	Los verás por consola en el paso 3

DOS COSAS DE LA PANTALLA QUE CONVIENE SABER ANTES DE BUSCARLAS

No hay una pestaña llamada *Partitions*. Las pestañas del tópico son *Overview*, *Messages*, *Consumers*, *Settings*, *Statistics* y *ACLs*. **La tabla de particiones está dentro de *Overview***, debajo de los recuadros de arriba.

Y no hay una columna *Leader*. En la columna *Replicas* de cada fila verás algo como `2, 3, 1`, y **el líder es el que está pintado de otro color**, que es el primero. La consola te lo dice con todas las letras y la pantalla te lo dice con un color. **La consola es más explícita aquí**, y conviene notarlo el día que discutas con alguien que solo mira el tablero.

Cómo se lee

Los números son **los mismos**. No hay un solo dato en esa pantalla que no hayas podido sacar de la línea de comandos.

Ahí está la primera mitad de la afirmación, y conviene decirla en voz alta. **El tablero no sabe nada que tú no puedas averiguar.** Es un cliente de Kafka, igual que la consola, igual que el programa Java del laboratorio 09. Le pregunta a los brokers y dibuja lo que le contestan.

LO QUE SÍ HACE MEJOR, Y ES HONESTO RECONOCERLO

El desbalance de particiones, o sea dos en cero y una en tres, en la consola son tres líneas que hay que sumar a ojo. En la pantalla es **una barra que está torcida**. Con tres particiones da igual. **En un tópico de doscientas, esa diferencia deja de ser una comodidad y pasa a ser la única forma práctica.**

VAS BIEN SI...

Para cada fila de la tabla de arriba puedes señalar el número en la pantalla y el número en la salida del paso 1, y son el mismo. **Anota una respuesta.** ¿Encontraste algún dato en el tablero que no supieras sacar por consola?

SI EL NAVEGADOR DICE QUE NO PUEDE CONECTAR

Comprueba dos cosas, en este orden.

Una. Que escribiste 8090 y no 8080.

Dos. Que el contenedor está arriba, con `docker ps --filter name=kafbat-ui`. Si no aparece, levántalo con `docker start kafbat-ui`.

SI EL DASHBOARD DICE OFFLINE · 1 CLUSTERS O LA LISTA SALE VACÍA

Kafbat arrancó antes que los brokers y se quedó con la primera respuesta. **Recarga la página.** Si sigue, reinícialo con `docker restart kafbat-ui` y dale medio minuto. **Esto ya es el laboratorio empezando**, porque acabas de ver que el tablero puede estar equivocado mientras el clúster está bien.

Lo que el tablero suma por ti

Qué vamos a hacer

Fabricar el problema más común de operación, que es un consumidor que se quedó atrás, y mirarlo por los dos lados.

CONCEPTO · GRUPO DE CONSUMIDORES

Varios consumidores que comparten un nombre de grupo y se reparten las particiones de un tópico.

Kafka guarda por dónde va el grupo, no cada consumidor.

CONCEPTO · LAG, O RETRASO

Cuántos mensajes hay escritos que el grupo todavía no leyó. Es la resta entre el offset final del tópico y el offset por donde va el grupo. **Es el número que se vigila en producción**, más que ningún otro.

Primero, un grupo que lee lo que hay y se va

```
docker exec kafka-broker-1 kafka-console-consumer \  
  --bootstrap-server kafka-broker-1:29092 \  
  --topic novatech.lab10.pedidos \  
  --group fiscalizacion --from-beginning --max-messages 3
```

--group fiscalizacion — **sin grupo no hay lag que medir**. Kafka solo guarda el avance de un grupo con nombre

--from-beginning — empieza por el mensaje más viejo que el tópico conserva

--max-messages 3 — lee los tres que hay y sale solo. **Sin esto se queda esperando para siempre**

LO QUE VAS A VER

```
{"pedido":1,"cliente":"ACME","monto":1500}  
{"pedido":2,"cliente":"ACME","monto":2500}  
{"pedido":3,"cliente":"ACME","monto":3500}  
Processed a total of 3 messages
```

Son los mismos tres que viste en la pestaña **Messages** del tablero. **El grupo acaba de guardar por dónde va.**

Ahora llegan cinco pedidos por la ventanilla de la calle

Y de paso esto es todo el REST Proxy que el recorrido necesita.

```
curl -s -w "\nHTTP %{http_code}\n" -X POST \
  -H "Content-Type: application/vnd.kafka.json.v2+json" \
  --data '{"records":[{"value":{"pedido":2001}}, {"value":{"pedido":2002}}, {"value":{"pedido":2003}}, {"value":{"pedido":2004}}, {"value":{"pedido":2005}}]}' \
  http://localhost:8082/topics/novatech.lab10.pedidos
```

-s — modo silencioso. Sin esto `curl` dibuja una barra de progreso encima de la respuesta

-w "\nHTTP %{http_code}\n" — imprime el código HTTP al final. **La respuesta sola no te lo dice**, y aquí importa

-X POST a `/topics/<tópico>` — producir por HTTP es **un POST y ya**. No hay sesión ni estado

-H "Content-Type: application/vnd.kafka.json.v2+json" — el tipo de contenido propio del REST Proxy. Dice **dos cosas a la vez**, que hablas su versión 2 y que los valores van en JSON

--data '{"records": [...]}' — los mensajes. **Un POST puede llevar varios**, y por eso van dentro de una lista

LO QUE VAS A VER

```
{
  "offsets": [
    {
      "partition": 0,
      "offset": 3,
      "error_code": null,
      "error": null
    },
    {
      "partition": 0,
      "offset": 4,
      "error_code": null,
      "error": null
    },
    {
      "partition": 0,
      "offset": 5,
      "error_code": null,
      "error": null
    },
    {
      "partition": 0,
      "offset": 6,
      "error_code": null,
      "error": null
    },
    {
      "partition": 0,
      "offset": 7,
      "error_code": null,
      "error": null
    }
  ],
  "key_schema_id": null,
  "value_schema_id": null
}
HTTP/200
```

Una entrada por mensaje, cada una con **la partición y el offset donde quedó**. Es exactamente lo que el broker le contestó al programa Java del laboratorio 09, el mismo dato, por HTTP.

Y AQUÍ UNA QUE SE PAGA CARA EN PRODUCCIÓN

Mira el `error` en `null` de cada entrada. **Un POST puede devolver 200 y traer errores adentro**, si alguno de los mensajes de la tanda falló. El código HTTP habla del POST. Los `error` hablan de cada mensaje. **Un sistema que solo mira el 200 pierde mensajes sin enterarse.**

El retraso, por la consola

```
docker exec kafka-broker-1 kafka-consumer-groups \
  --bootstrap-server kafka-broker-1:29092 \
  --describe --group fiscalizacion
```

LO QUE VAS A VER

```
Consumer group 'fiscalizacion' has no active members.
```

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG
CONSUMER-ID	HOST	CLIENT-ID			
fiscalizacion	novatech.lab10.pedidos	0	3	8	5
-	-				-
fiscalizacion	novatech.lab10.pedidos	1	0	0	0
-	-				-
fiscalizacion	novatech.lab10.pedidos	2	0	0	0
-	-				-

Lo que dice

Qué significa

`has no active members`

Nadie está consumiendo ahora mismo. **El grupo existe porque dejó su marca,** pero está parado

`CURRENT-OFFSET 3`

Por dónde va el grupo en la partición llena

`LOG-END-OFFSET 8`

Hasta dónde llegó el tópico

`LAG 5`

Los cinco que entraron por HTTP y nadie leyó

Y la misma verdad en el tablero

En Kafbat, entra a **Consumers** y luego a `fiscalizacion`.

Lo que muestra el tablero

Lo que dijo la consola

State en EMPTY

`has no active members`

Members en 0

Los guiones de las columnas `CONSUMER-ID`, `HOST` y `CLIENT-ID`

Assigned Partitions en 3

Que la tabla trae tres filas, una por partición

Un solo número grande, **Total lag 5**

La consola no lo da sumado. Da tres filas, y el 5 hay que encontrarlo entre ceros

EL NÚMERO GRANDE SE LLAMA TOTAL LAG

Está arriba, en la fila de recuadros, junto a *State* y *Members*. Más abajo hay una tabla con una fila por tópico y una columna **Consumer lag**, que aquí dice 5 también. **Son el mismo número por un solo tópico.** Con varios tópicos, el de arriba es la suma de los de abajo.

LA SEGUNDA MITAD DE LA AFIRMACIÓN, Y ES LA PARTE HONESTA

El tablero sí hace algo mejor. No sabe nada distinto, porque los números son los mismos. Pero te los suma y te los pone en la cara. Con tres particiones da igual. **Con doscientas es la diferencia entre ver el problema y no verlo.**

VAS BIEN SI...

La consola dice `LAG 5` en una partición y 0 en las otras dos, y el tablero dice **Total lag 5** en un solo número.

Anota una respuesta. ¿De dónde sacó el tablero ese 5, si la consola no lo da sumado?

SI EL POST DEVUELVE `HTTP 415`

Es el error más común de este paso, y está medido. Pasa cuando escribes `Content-Type: application/json` a secas.

El REST Proxy exige **su propio tipo de contenido**, que es `application/vnd.kafka.json.v2+json`. Cópialo entero, con el `vnd.kafka` y el `v2`. **El 415 quiere decir «no hablo ese formato»**, y es del proxy, no de Kafka.

SI EL `--DESCRIBE` DICE QUE EL GRUPO NO EXISTE

No corriste el consumidor con `--group fiscalizacion`, o escribiste otro nombre. **Un grupo se crea al usarlo**, no antes. Lista los que hay con `--list` en vez de `--describe --group`.

SI EL `LAG` TE SALE 0 EN TODO

Corriste el `--describe` antes del POST, o el consumidor volvió a leer después. Vuelve a mandar el POST de cinco y consulta de nuevo, **sin consumir en medio**.

PASO 4 DE 5

Y ahora se cae el tablero

Qué vamos a hacer

Éste es el paso que cierra el laboratorio, y es la razón de que exista. Vamos a apagar la interfaz **a propósito** y a ver qué se rompe.

PREDICE ANTES DE EJECUTAR, Y ESCRÍBELO

Con Kafbat apagado, ¿el clúster sigue funcionando? ¿Y el REST Proxy? ¿Y el grupo `fiscalizacion` pierde su avance? **Contesta las tres antes de teclear.** El paso vale la mitad si lees la respuesta antes de haber apostado.

El comando

```
docker stop kafbat-ui
```

docker stop — apaga el contenedor de forma ordenada. **No borra nada**, porque los datos de Kafbat viven en el clúster y no en él

kafbat-ui — el nombre del contenedor de la interfaz. Es uno de los cinco de este laboratorio

LO QUE VAS A VER

```
kafbat-ui
```

Una sola línea. Docker imprime el nombre de lo que apagó, y nada más. **No hay confirmación ni advertencia.**

Recarga `http://localhost:8090` en el navegador y no responde. Y por si queda duda de que está caído y no lento:

```
curl -sS --max-time 5 http://localhost:8090/
```

-sS — silencioso **pero mostrando los errores**. La `S` mayúscula es la que deja pasar el mensaje de fallo

--max-time 5 — se rinde a los cinco segundos. **Aquí no hace falta**, porque el fallo es inmediato, pero es la costumbre correcta

LO QUE VAS A VER

```
curl: (7) Failed to connect to localhost port 8090 after 0 ms: Couldn't connect to server
```


Dice *Failed to connect*, y no 404 ni 500. **No hay nadie escuchando en ese puerto.** El (7) es el código de `curl` para «no pude conectar».

LA DISTINCIÓN QUE HAY QUE LLEVARSE

Un **404** o un **500** quieren decir que alguien contestó. Un **(7)** quiere decir que no hay nadie. **Son diagnósticos distintos y llevan a sitios distintos**, y confundirlos a las tres de la mañana cuesta media hora.

Las tres preguntas de la predicción, contestadas con comandos

```
docker exec kafka-broker-1 kafka-get-offsets \  
  --bootstrap-server kafka-broker-1:29092 \  
  --topic novatech.lab10.pedidos --time -1
```

LO QUE VAS A VER

```
novatech.lab10.pedidos:0:8  
novatech.lab10.pedidos:1:0  
novatech.lab10.pedidos:2:0
```

```
docker exec kafka-broker-1 kafka-consumer-groups \  
  --bootstrap-server kafka-broker-1:29092 \  
  --describe --group fiscalizacion
```

LO QUE VAS A VER

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG
CONSUMER-ID	HOST	CLIENT-ID			
fiscalizacion	novatech.lab10.pedidos	0	3	8	5
-	-				-
fiscalizacion	novatech.lab10.pedidos	1	0	0	0
-	-				-
fiscalizacion	novatech.lab10.pedidos	2	0	0	0
-	-				-

```
curl -s -o /dev/null -w "REST Proxy: HTTP %{http_code}\n" http://localhost:8082/topics
```

-o /dev/null — tira el cuerpo de la respuesta. **Aquí solo interesa el código**, no la lista de tópicos

LO QUE VAS A VER

```
REST Proxy: HTTP 200
```

Cómo se lee

No se movió nada. Los ocho mensajes siguen ahí, el lag sigue siendo 5, y la ventanilla de la calle sigue atendiendo.

El clúster no se enteró de que el tablero se cayó, porque el tablero nunca fue parte del clúster. Era un cliente que preguntaba y dibujaba.

Y para volver

```
docker start kafbat-ui
```

Imprime otra vez `kafbat-ui`, y ya está. **Medido, vuelve a responder en 5 segundos.** Recarga el navegador y está todo igual, porque no había nada que recuperar.

LO LOGRASTE SI...

Con el tablero apagado, los tres comandos te dieron ocho mensajes, lag 5 y `HTTP 200`. Y al volver a encenderlo, la pantalla mostró lo mismo que antes sin que tú hicieras nada.

Compara con tu predicción. ¿Acertaste las tres?

SI `DOCKER START KAFBAT-UI` DICE `NO SUCH CONTAINER`

Corriste `docker rm` en vez de `docker stop`, o algo lo borró. Levántalo con el compose del lab:

```
cd infra && docker compose up -d kafbat-ui && cd ..
```

SI AL VOLVER EL TABLERO SE QUEDA CARGANDO

Dale medio minuto. Kafbat vuelve a preguntarle todo al clúster desde cero al arrancar, y eso tarda unos segundos. **Si a los dos minutos sigue igual,** compruébalo por su propia API, que es más barata que la pantalla:

```
curl -s http://localhost:8090/api/clusters
```

Los tres comandos de las tres de la mañana

Qué vamos a hacer

Cerrar el laboratorio con lo único que de verdad se lleva. **Escribir, sin mirar, los tres comandos con los que se responde un incidente cuando no hay tablero.**

No hay nada nuevo que ejecutar. Son tres que ya usaste hoy.

La pregunta	El comando
¿El clúster está vivo?	<code>kafka-get-offsets --bootstrap-server ... --topic ... --time -1</code>
¿El tópicos está sano?	<code>kafka-topics --bootstrap-server ... --describe --topic ...</code>
¿Quién se está atrasando?	<code>kafka-consumer-groups --bootstrap-server ... --describe --group ...</code>

POR QUÉ ÉSTOS Y NO OTROS

El primero es el más barato que existe para saber si hay alguien al otro lado. Si contesta, el clúster está vivo.

El segundo dice si las réplicas están al día. Una partición con menos réplicas en ISR que en `Replicas` es un problema que todavía no se ve.

El tercero es el que encuentra el problema real nueve de cada diez veces. **El clúster suele estar bien y el que se quedó atrás es un consumidor.**

LO LOGRASTE SI...

Puedes escribir los tres de memoria, con su `--bootstrap-server`, sin abrir esta guía. **Ése es el laboratorio.** Todo lo demás de hoy fue para que entiendas por qué estos tres y no la pantalla.

Lo que aprendiste

1 El tablero es un cliente, no el clúster.

Todo lo que muestra se lo preguntó a los brokers, igual que la consola y que el programa Java del laboratorio 09. Si muestra algo raro, la primera pregunta es si el raro es el clúster o es el tablero.

2 Un tablero caído no es un clúster caído, y confundirlos cuesta una madrugada.

La forma de distinguirlos es un `kafka-get-offsets` desde cualquier broker. Y en la respuesta de `curl`, un 404 o un 500 quieren decir que alguien contestó. Un `(7)` quiere decir que no hay nadie.

3 Lo que se mira en el tablero se arregla en la consola.

Mirar y actuar son dos cosas distintas. A las tres de la mañana solo vas a tener la segunda. Por eso el tablero suma mejor, y aun así no reemplaza a los tres comandos del paso 5.

4 REST Proxy es para el sistema que no puede tener un cliente nativo.

Si puede, debe. Cada mensaje que pasa por el proxy pasa por un intermediario más, que hay que dimensionar, vigilar y mantener arriba. Y su respuesta puede traer un 200 con errores adentro, así que se revisan los dos niveles.

Para profundizar

Todo esto estaba en el recorrido de clase y salió por tiempo. Aquí va desarrollado, con su comando.

A · REST Proxy, endpoint por endpoint

El recorrido usó un solo POST. La API tiene más.

```
curl -s http://localhost:8082/topics
curl -s http://localhost:8082/topics/novatech.lab10.pedidos
```

LO QUE SALE DEL PRIMERO

```
["novatech.lab10.pedidos"]
```

LO QUE SALE DEL SEGUNDO · RECORTADO A LAS PARTICIONES

```
{
  "name": "novatech.lab10.pedidos",
  "configs": { ... },
  "partitions": [
    {
      "partition": 0,
      "leader": 2,
      "replicas": [
        { "broker": 2, "leader": true, "in_sync": true },
        { "broker": 3, "leader": false, "in_sync": true },
        { "broker": 1, "leader": false, "in_sync": true }
      ]
    },
    { "partition": 1, "leader": 3, ... },
    { "partition": 2, "leader": 1, ... }
  ]
}
```

Lo que hay que mirar. El bloque `configs` viene entero y son treinta y tantas propiedades, así que arriba está recortado. Lo importante es `partitions`. Ahí están las particiones, sus líderes y sus réplicas, con un `in_sync` por réplica. **Son los mismos datos del `--describe` del paso 1, por HTTP.** Tres formas de preguntar lo mismo, o sea la consola, el tablero y el proxy.

Y los envoltorios del curso, que son `curl` por dentro y no dependen de nada instalado:

```
rest-cli/rest-list-topics.sh
rest-cli/rest-produce.sh novatech.lab10.pedidos '{"pedido":3001,"cliente":"Courier-Y"}'
```

B · Consumir por HTTP, que es lo que de verdad cuesta

Producir por HTTP era un POST. **Consumir es un ciclo de vida de cuatro pasos**, y esa asimetría es el bloque más instructivo del laboratorio.

```
crear instancia → suscribir → poll (dos veces) → borrar instancia
```

```
rest-cli/rest-consume.sh novatech.lab10.pedidos
```

LO QUE SALE · RECORTADO

```
[REST Consume] Ciclo completo del consumer HTTP
1. Crear instancia (ci-75005) en grupo novatech-rest-group...
{"instance_id":"ci-75005","base_uri":"http://kafka-rest:8082/consumers/novatech-rest-group/instances/ci-75005"}
2. Suscribirse al tópico novatech.lab10.pedidos...
3. Primer poll (inicializa la suscripción, suele venir vacío)...
4. Segundo poll (trae los mensajes)...
[{"topic":"novatech.lab10.pedidos","key":null,"value":
{"pedido":1,"cliente":"ACME","monto":1500},"partition":0,"offset":0},
{"topic":"novatech.lab10.pedidos","key":null,"value":{"pedido":2001},"partition":0,"offset":3}, ... ]
5. Borrar la instancia...

✓ Ciclo de consumo HTTP completado
```

Lo que hay que mirar. Dos cosas.

El primer poll viene vacío. No es un fallo. La primera llamada es la que activa la suscripción, y recién la segunda trae datos.

Hay que borrar la instancia al terminar. Si no, queda ocupando su asignación de particiones en el servidor hasta que expire.

Y AHÍ ESTÁ EL PORQUÉ DE TODA LA ASIMETRÍA

Producir no necesita memoria, porque el mensaje se va y listo. **Consumir sí, porque Kafka tiene que recordar por dónde vas.** HTTP no tiene memoria entre llamadas, así que el proxy la guarda por ti. Por eso hay que crear y borrar algo.

La pregunta que vale. ¿Qué pasa con las instancias de consumer que un sistema mal escrito crea y nunca borra?

C · El desafío del socio que solo hace `curl`

Diséñale a un socio externo una secuencia de tres o cuatro `curl` que liste los tópicos, produzca dos pedidos, cree un consumer, lea y limpie la instancia. Documentala en el reporte.

Es el ejercicio que convierte este laboratorio en algo que puedes entregarle a alguien.

D · Interoperabilidad, HTTP y nativo sobre el mismo tópico

```
rest-cli/rest-produce.sh novatech.lab10.pedidos '{"pedido":9999,"origen":"http"}'
docker exec kafka-broker-1 kafka-console-consumer \
  --bootstrap-server kafka-broker-1:29092 \
  --topic novatech.lab10.pedidos --from-beginning --max-messages 10
```

Lo que hay que mirar. El cliente nativo lee el mensaje que entró por HTTP, sin saber ni poder saber por dónde entró. **Es la misma lección del laboratorio 09.** El broker no distingue clientes.

E · La API del propio tablero

Kafbat tiene su propia API REST, y es la que usa su pantalla.

```
curl -s http://localhost:8090/api/clusters
```

LO QUE SALE

```
[{"name":"novatech-cluster","defaultCluster":null,"status":"ONLINE","lastError":null,"brokerCount":3,"onlinePartitionCount":53,"topicCount":2,"bytesInPerSec":null,"bytesOutPerSec":null,"readOnly":false,"version":"4.2-IV1","features":["TOPIC_DELETION","CLIENT_QUOTA_MANAGEMENT","FTS_ENABLED"],"controller":"KRAFT"}]
```

Lo que hay que mirar. El `"controller":"KRAFT"` y el `"version":"4.2-IV1"` son los mismos datos del laboratorio 01.

Y DOS NÚMEROS QUE SORPRENDEN, CON SU EXPLICACIÓN

Dice `topicCount: 2` y `onlinePartitionCount: 53`, cuando tú solo creaste un tópico de tres particiones.

El segundo tópico es `__consumer_offsets`, el interno donde Kafka guarda por dónde va cada grupo.

Tiene 50 particiones, y 50 más 3 son 53. Apareció cuando el grupo `fiscalizacion` guardó su avance en el paso 3. **Si corres esto antes del paso 3, te va a dar 1 y 3.** Ése es el tópico que hace que los offsets sobrevivan a que apagues el consumidor.

UNA IDEA QUE VALE PARA PRODUCCIÓN

Si quieres vigilar el clúster desde un tablero propio o un monitor, **no hace falta que una persona mire esta pantalla.** Se le puede preguntar a esta API.

F · El reporte del laboratorio

`plantillas/reporte-entregable.md` recorre las actividades con sus preguntas. Las respuestas de referencia están en `soluciones/reporte-resuelto.md`.

Antes de cerrar

DEJA EL TERRENO LIMPIO

```
bin/90-test-lab.sh  # ver el estado del laboratorio
bin/stop-lab.sh    # apagar los contenedores
```

`bin/stop-lab.sh` solo detiene los contenedores. No borra nada, y los volúmenes siguen ahí mientras el lab esté detenido.

Ojo al reanudar. `bin/start-lab.sh` reconstruye el laboratorio desde cero, así que los tópicos y mensajes que creaste no sobreviven. El lab vuelve a sembrar los suyos. **Si creaste algo propio, anótalo antes.** Para borrarlo todo ahora mismo, `bin/reset-lab.sh`, que no pide confirmación.

LO QUE DICE EL VALIDADOR SI TODO ESTÁ EN ORDEN

```
Validador del Lab 10 – estado actual
✓ los 3 brokers están healthy
✓ REST Proxy responde en :8082/topics
✓ el tópico novatech.lab10.pedidos es visible vía REST
✓ kafbat UI responde UP en :8090

✓ LAB EN BUEN ESTADO (4 verificaciones OK)
```

LO QUE TE LLEVAS

Si mañana a las tres de la mañana te llaman por Kafka y solo tienes un SSH a un servidor sin navegador, **¿qué tres comandos escribes primero?**

Hoy los escribiste todos, y además viste por qué el tablero no va a estar. **El pizarrón se puede borrar y el restaurante sigue atendiendo.** Lo que no se puede borrar es lo que tú sabes escribir.